



LOCKE & KEY

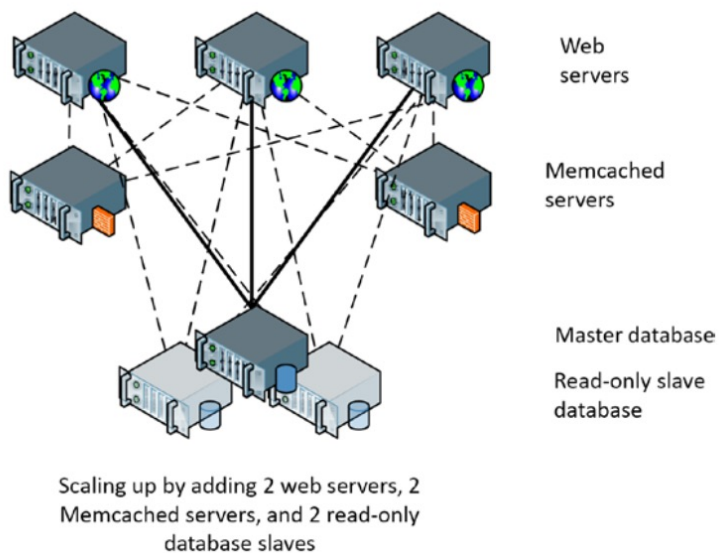




Plan

- ▣ Qu'est ce que REDIS ?
- ▣ Les fondamentaux
- ▣ Comment utiliser REDIS ?
- ▣ La haute disponibilité

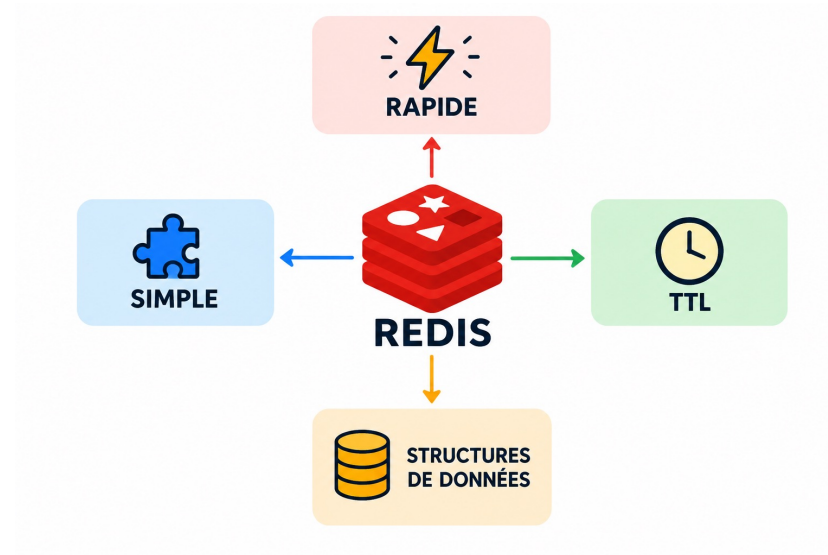
Pourquoi Redis ?



Créé en 2009 par Salvatore SANFILIPPO

- *Developeur dans de solution analytique web en temps réel*
- *Mysql ne pouvait pas fournir les performances nécessaire*
- *Il a donc créer sa propre solution REDIS pour visualiser en temps réel les logs applicatifs*
- *En juin 2009, Redis a été développée en production sous l'application [LLOOGG](#)*
- *En 2010, il est embauché par VMWARE pour développer REDIS*

Qu'est ce que Redis ?



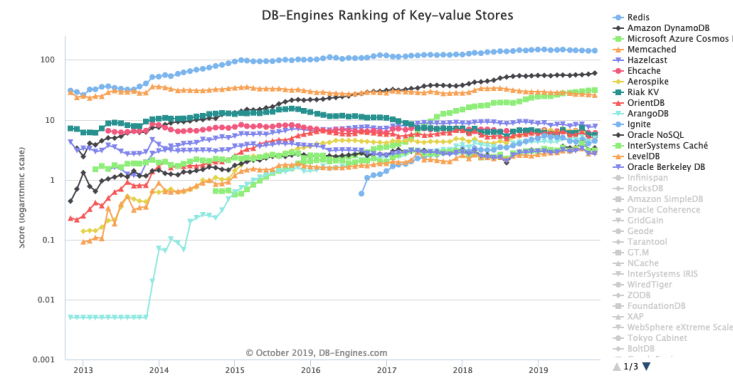
- *Salvatore SANFILIPPO définit Redis de la manière suivante :*
 - « I see Redis definitely more as a flexible tool than as a solution specialized to solve a specific problem: his mixed soul of cache, store, and messaging server shows this very well »

Qui utilise REDIS ?



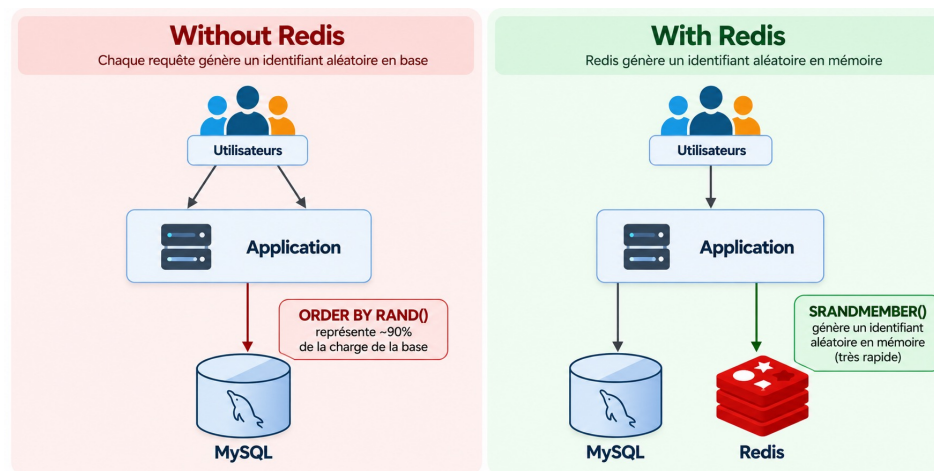
Redis aujourd'hui ?

- D'après DB-Engines, Redis est classée comme la 1ère base de données clé/valeur.



- En **mars 2024**, Redis abandonne la licence BSD au profit de licences *Source Available*, entraînant la création de **Valkey**, un fork open source sous licence BSD porté par la Linux Foundation.
- Depuis **Redis 8 en 2025**, Redis propose également la licence **AGPLv3**, reconnue open source, et est donc de nouveau disponible sous une licence open source.
- **Pour un nouveau projet où l'objectif est de privilégier un logiciel libre, Valkey mérite d'être privilégié dans l'étude de choix,**

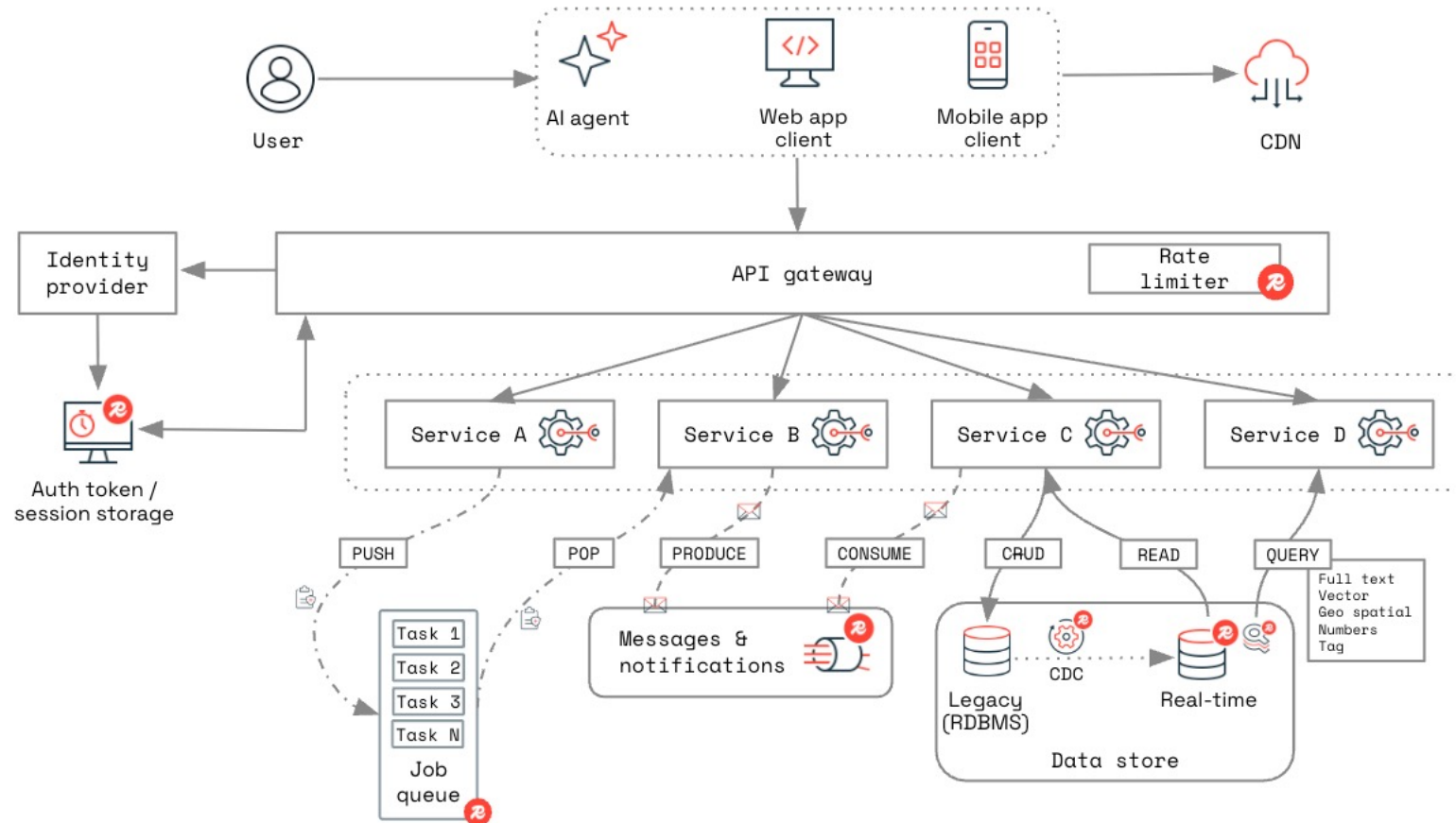
Redis pour quels usages : The Guardian



- Pour son application d'analyse des dépenses des parlementaires, The Guardian devait sélectionner aléatoirement une page non encore examinée.
- **Avant** : sélection aléatoire en SQL avec `ORDER BY RAND() LIMIT 1` → traitement inefficace.
- **Avec Redis** : représentation des pages sous forme de **Sets**, calcul des pages restantes avec `SDIFFSTORE`, puis sélection avec `SRANDMEMBER`.
- **Valeur ajoutée** : Redis fournit une structure de données et des opérations directement adaptées au problème.

- Cache : Accélérer l'accès aux données et soulager la base principale
 - Sessions : Stocker les sessions utilisateurs
 - Compteurs : Vues, likes, téléchargements, quotas...
 - Classements : Leaderboards grâce aux Sorted Sets
 - Données temporaires : Tokens, codes de validation, données avec TTL
 - Messaging : Pub/Sub, Streams, files de messages
 - **Accéder ou traiter certaines données très rapidement sans solliciter systématiquement le stockage principal.**
-

Les cas d'utilisation aujourd'hui de Redis

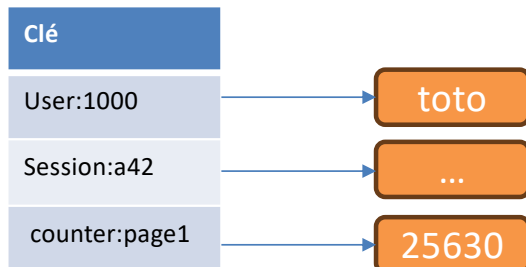
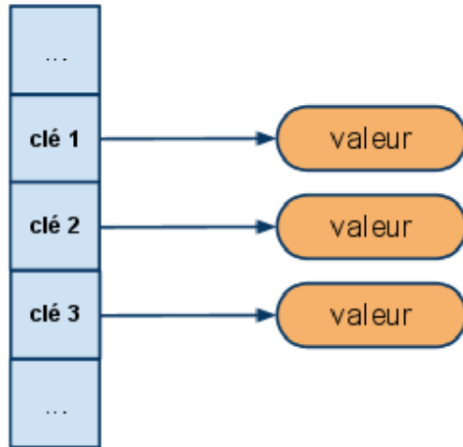


Vous utilisez donc probablement Redis tous les jours sans le savoir.

Mais pourquoi est-il particulièrement adapté à tous ces usages ?

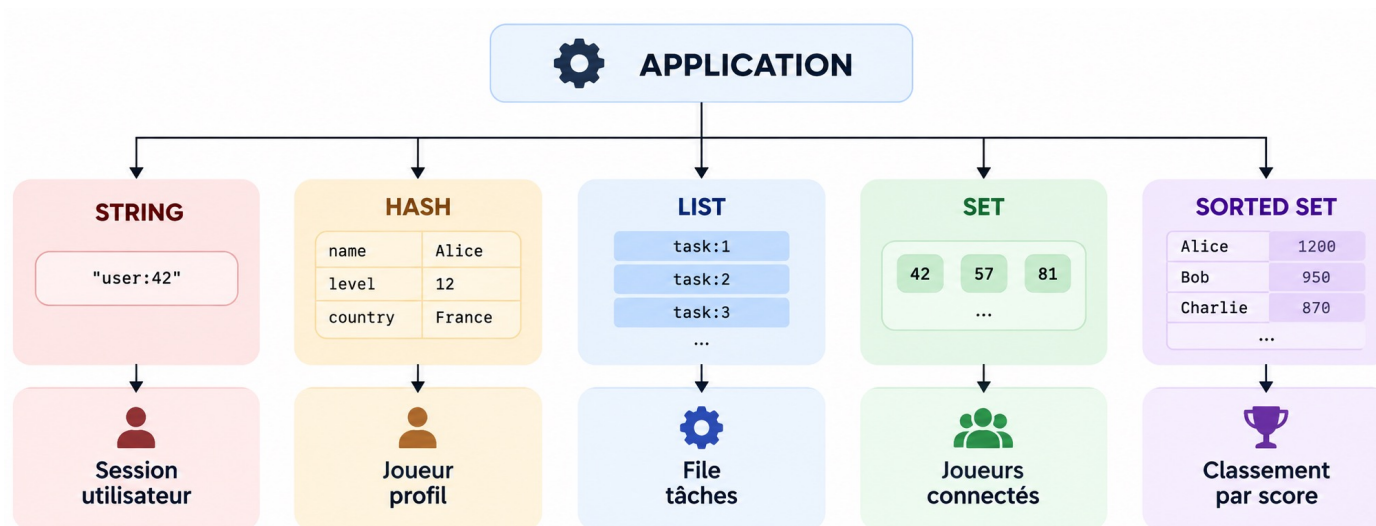
Pour le comprendre, commençons par son modèle de données : le modèle clé-valeur.

Le modèle clé-valeur



- REDIS est l'acronyme de REmote Dictionary Server : serveur de dictionnaire distant.
- C'est un dictionnaire qui associe à une clé une et une seule valeur.
- Il n'y pas de table où de schéma prédéfini.
- Conserve les données en mémoire et peut être configuré pour conserver les données sur disque.

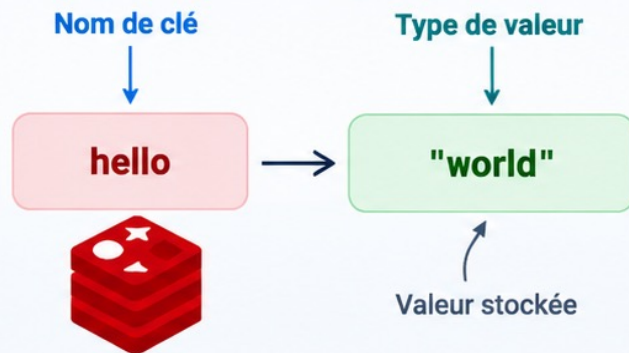
Une clé Redis ne pointe pas simplement vers une valeur : elle peut pointer vers une structure adaptée à un usage.



Redis – Structure de données : **STRING**

La structure String permet de stocker une chaîne de caractères, un entier ou un nombre flottant.

1 Concept



2 Ce qu'elle contient



Chaînes de caractères

"hello world"



Entiers

42



Nombres flottants

3.14

3 Capacités de lecture/écriture



Opérer sur la chaîne complète
(lecture, écriture, remplacement)



Opérer sur une partie de la chaîne
(extraire ou modifier une portion)



Incrémenter / décrémenter des entiers
(de manière atomique)



Incrémenter / décrémenter des nombres flottants
(de manière atomique)

4 Commandes principales

Commande	Description	Exemple
GET key	Récupère la valeur associée à la clé.	GET hello
SET key value	Définit la valeur associée à la clé.	SET hello "world"
DEL key	Supprime la valeur associée à la clé (fonctionne pour tous les types).	DEL hello
INCR key	Incrémente un entier (de 1, opération atomique).	INCR counter
DECR key	Décrémente un entier (de 1, opération atomique).	DECR counter
INCRBYFLOAT key value	Incrémente un nombre flottant.	INCRBYFLOAT score 0.5



Cas d'utilisation de **STRING** : gestion des sessions utilisateur

Le type String permet de stocker une valeur simple (chaîne de caractères, entier ou nombre flottant).
Il est idéal pour stocker des informations simples comme un token de session, un compteur ou une configuration.

1 Le besoin

Notre application doit gérer les sessions utilisateur.

Après authentification, on génère un token de session et on le stocke dans Redis avec une durée de vie limitée (ex. 30 minutes).

Nous voulons :

- stocker le token de session,
- récupérer rapidement la session,
- supprimer automatiquement la session après expiration,
- mettre à jour la durée de vie si l'utilisateur est actif.



2 La structure

On utilise un **STRING** où :

- la clé est l'identifiant de session (ex. `session:12345`),
- la valeur est le token (chaîne de caractères),
- une durée de vie (TTL) est définie pour l'expiration automatique.



3 Exemple d'utilisation



Stocker un token de session (avec expiration)

```
SET session:12345 "a3f5e8d9c2f1..." EX 1800  
# Stocke le token avec une durée de vie de 30 min
```



Récupérer le token de session

```
GET session:12345  
# Retourne "a3f5e8d9c2f1..."
```



Mettre à jour la durée de vie (si l'utilisateur est actif)

```
EXPIRE session:12345 1800  
# Prolonge l'expiration de 30 min
```



Supprimer la session (déconnexion)

```
DEL session:12345  
# Supprime le token de session
```

4 Résultat

Stockage du token avec expiration

```
127.0.0.1:6379> SET session:12345  
"a3f5e8d9c2f1..." EX 1800  
OK  
  
127.0.0.1:6379> TTL session:12345  
(integer) 1800
```

Récupération du token

```
127.0.0.1:6379> GET session:12345  
"a3f5e8d9c2f1..."
```

Après expiration

```
# Après 30 minutes  
127.0.0.1:6379> GET session:12345  
(nil)  
# Le token a expiré automatiquement
```



Pourquoi utiliser un **STRING** ?



Stockage simple et rapide de petites données



Peut contenir des chaînes, entiers ou nombres flottants



Support des opérations sur tout ou partie de la chaîne (incrément/décément pour les nombres)



Idéal pour les tokens, compteurs, configurations, flags, ...

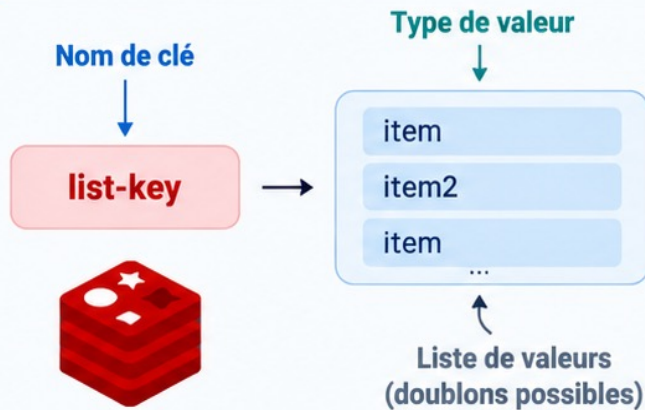


Support natif d'une durée de vie (TTL) pour l'expiration

Redis – Structure de données : LIST

La structure List permet de stocker une liste ordonnée de valeurs (des doublons sont possibles).

1 Concept



2 Ce qu'elle contient

- 1 Premier élément
"item"
- 2 Deuxième élément
"item2"
- 3 Troisième élément
"item"
- ⋮

3 Capacités de lecture/écriture

- ✓ Ajouter des éléments aux deux extrémités (gauche et droite)
- ✓ Lire un ou plusieurs éléments (par position ou par plage)
- ✓ Supprimer des éléments (à gauche ou à droite)
- ✓ Conserver l'ordre d'insertion

4 Commandes principales

Commande	Description	Exemple
RPUSH key value	Ajoute une valeur à la fin (côté droit) de la liste.	RPUSH list-key "item"
LPUSH key value	Ajoute une valeur au début (côté gauche) de la liste.	LPUSH list-key "item"
LRANGE key start stop	Récupère une plage de valeurs de la liste.	LRANGE list-key 0 2
LINDEX key index	Récupère un élément à une position donnée.	LINDEX list-key 1
LPOP key	Supprime et retourne le premier élément (côté gauche).	LPOP list-key
RPOP key	Supprime et retourne le dernier élément (côté droit).	RPOP list-key



Cas d'utilisation de LIST : une file de tâches

Le type List permet de stocker une séquence de valeurs ordonnées (avec doublons possibles). Il est idéal pour implémenter une file de tâches (queue) dans laquelle les éléments sont ajoutés et traités dans l'ordre.

1 Le besoin

Notre application doit traiter des tâches de manière asynchrone (ex. traitement de fichiers, envoi d'emails, génération de rapports, ...).

Nous voulons :

- ajouter des tâches à traiter,
- les traiter dans l'ordre d'arrivée (FIFO),
- distribuer les tâches entre plusieurs workers,
- gérer facilement la récupération des tâches.



2 La structure

On utilise une LIST où :

- la clé représente la file de tâches,
- chaque élément est une tâche (ex. un identifiant, un message JSON, ...),
- l'ordre est conservé (FIFO : premier entré, premier sorti),
- les doublons sont autorisés.



3 Exemple d'utilisation

+ Ajouter une tâche à la fin de la file

```
RPUSH tasks:queue "traiter_fichier_01"
RPUSH tasks:queue "envoyer_email_02"
```

🔍 Voir toutes les tâches en attente

```
LRANGE tasks:queue 0 -1
```

▶ Récupérer et supprimer la première tâche

```
LPOP tasks:queue
# Retourne "traiter_fichier_01"
```

☰ Récupérer une tâche sans la supprimer

```
LINDEX tasks:queue 0
# Retourne la prochaine tâche
```

4 Résultat

File après ajout de tâches

```
127.0.0.1:6379> LRANGE tasks:queue 0 -1
1) "traiter_fichier_01"
2) "envoyer_email_02"
3) "generer_rapport_03"
4) "nettoyer_donnees_04"
```

Récupération de la première tâche

```
127.0.0.1:6379> LPOP tasks:queue
"traiter_fichier_01"

127.0.0.1:6379> LRANGE tasks:queue 0 -1
1) "envoyer_email_02"
2) "generer_rapport_03"
3) "nettoyer_donnees_04"
```

Accès à une tâche sans la supprimer

```
127.0.0.1:6379> LINDEX tasks:queue 0
"envoyer_email_02"
```

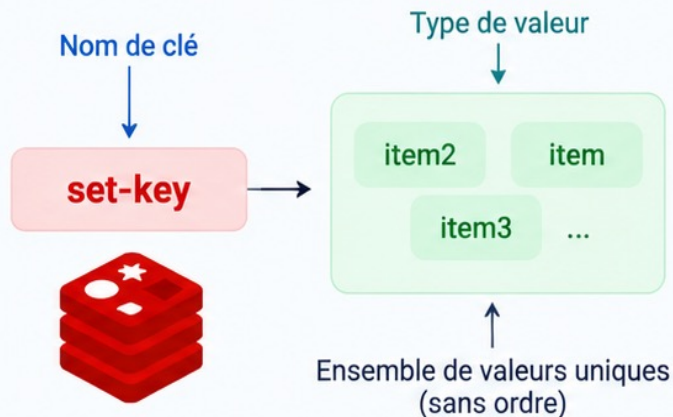
💡 Pourquoi utiliser une LIST ?

- ✓ Gestion d'une file de tâches (queue) simple et efficace (FIFO)
- ✓ Préservation de l'ordre d'ajout
- ✓ Doublons autorisés si nécessaire
- ✓ Idéal pour les traitements asynchrones (workers, background jobs, pipelines, ...)

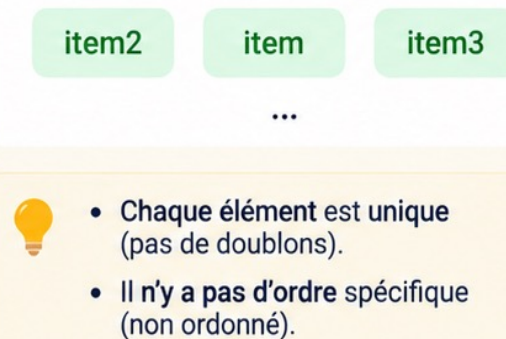
Redis – Structure de données : SET

La structure **Set** permet de stocker un ensemble de valeurs uniques (pas de doublons) et sans ordre défini.

1 Concept



2 Ce qu'elle contient



3 Capacités de lecture/écriture

- ✓ **Ajouter des éléments**
(si l'élément n'existe pas déjà)
- ✓ **Vérifier l'appartenance d'un élément**
(test rapide)
- ✓ **Récupérer tous les éléments de l'ensemble**
- ✓ **Supprimer un élément**
(s'il existe)
- ✓ **Opérations sur ensembles**
(union, intersection, différence, ...)

4 Commandes principales

Commande	Description	Exemple
<code>SADD key member [member ...]</code>	Ajoute un ou plusieurs éléments à l'ensemble.	<code>SADD set-key item item2 item3</code>
<code>SMEMBERS key</code>	Retourne tous les éléments de l'ensemble.	<code>SMEMBERS set-key</code>
<code>SISMEMBER key member</code>	Vérifie si un élément appartient à l'ensemble.	<code>SISMEMBER set-key item2</code>
<code>SREM key member [member ...]</code>	Supprime un ou plusieurs éléments de l'ensemble.	<code>SREM set-key item</code>
<code>SCARD key</code>	Retourne le nombre d'éléments dans l'ensemble.	<code>SCARD set-key</code>
<code>SINTER key1 key2</code>	Retourne l'intersection entre deux ensembles.	<code>SINTER set1 set2</code>
<code>SUNION key1 key2</code>	Retourne l'union entre deux ensembles.	<code>SUNION set1 set2</code>
<code>SDIFF key1 key2</code>	Retourne la différence entre deux ensembles (éléments de key1 non présents dans key2).	<code>SDIFF set1 set2</code>



Cas d'utilisation de SET : gestion des utilisateurs connectés

Le type Set permet de stocker un ensemble de valeurs uniques (pas de doublons).
Il est idéal pour gérer des groupes d'éléments et tester rapidement l'appartenance d'un élément.

1 Le besoin

Notre application doit suivre les utilisateurs actuellement connectés au site.

Nous voulons :

- ajouter un utilisateur lorsqu'il se connecte,
- éviter les doublons,
- savoir rapidement si un utilisateur est connecté,
- obtenir la liste des utilisateurs connectés,
- supprimer un utilisateur lorsqu'il se déconnecte.



Déconnecté



Connecté

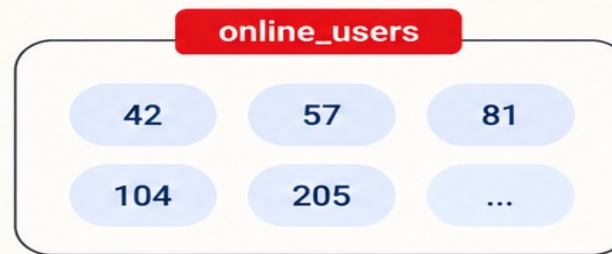


Déconnecté

2 La structure

On utilise un SET où :

- la clé représente l'ensemble des utilisateurs connectés,
- chaque élément est l'identifiant d'un utilisateur,
- les valeurs sont uniques (pas de doublons),
- l'ordre n'est pas défini.



Ensemble d'identifiants d'utilisateurs connectés
(valeurs uniques, ordre non défini)

3 Exemple d'utilisation



Ajouter un utilisateur connecté

```
SADD online_users 42  
# Ajoute l'utilisateur 42 à l'ensemble
```



Vérifier si un utilisateur est connecté

```
SISMEMBER online_users 42  
# Retourne 1 si l'utilisateur est connecté
```



Récupérer la liste des utilisateurs connectés

```
SMEMBERS online_users  
# Retourne tous les utilisateurs connectés
```



Supprimer un utilisateur connecté

```
SREM online_users 42  
# Supprime l'utilisateur 42 de l'ensemble
```



Compter le nombre d'utilisateurs connectés

```
SCARD online_users  
# Retourne le nombre d'éléments
```

4 Résultat

Liste des utilisateurs connectés

```
127.0.0.1:6379> SMEMBERS online_users  
1) "42"  
2) "57"  
3) "81"  
4) "104"  
5) "205"
```

Vérifier l'appartenance

```
127.0.0.1:6379> SISMEMBER online_users 42  
(integer) 1  
  
127.0.0.1:6379> SISMEMBER online_users 999  
(integer) 0
```

Nombre d'utilisateurs connectés

```
127.0.0.1:6379> SCARD online_users  
(integer) 5
```



Pourquoi utiliser un SET ?



Valeurs uniques (pas de doublons)



Test d'appartenance très rapide



Opérations ensemblistes (union, intersection, différence)

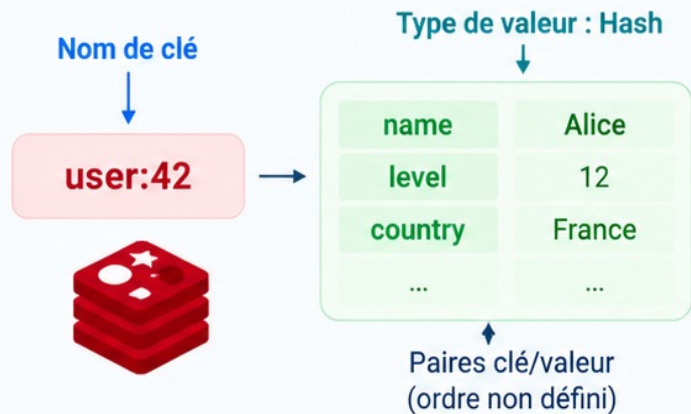


Idéal pour gérer des groupes d'éléments
(utilisateurs connectés, tags, groupes, ...)

Redis – Structure de données : HASH

La structure Hash permet de stocker un ensemble de paires clé/valeur sous une même clé Redis (idéal pour représenter un objet avec plusieurs attributs).

1 Concept



2 Ce qu'elle contient

name	Alice
level	12
country	France
email	alice@example.com
...	...



- Chaque **champ** (sous-clé) est une chaîne de caractères et possède sa **propre valeur**.
- L'ordre des **champs** n'est pas défini.

3 Capacités de lecture/écriture



Ajouter ou modifier un champ
(un ou plusieurs champs)



Récupérer la valeur d'un champ
(par sous-clé)



Récupérer l'ensemble des champs
(avec leurs valeurs)



Supprimer un champ
(s'il existe)

4 Commandes principales

Commande	Description	Exemple
HSET key field value [field value ...]	Ajoute ou met à jour un champ dans le hash (un ou plusieurs).	HSET user:42 name "Alice" level "12"
HGET key field	Récupère la valeur d'un champ donné.	HGET user:42 name
HMGET key field [field ...]	Récupère les valeurs de plusieurs champs.	HMGET user:42 name level
HGETALL key	Récupère tous les champs et leurs valeurs.	HGETALL user:42
HDEL key field [field ...]	Supprime un ou plusieurs champs.	HDEL user:42 country
HEXISTS key field	Vérifie si un champ existe.	HEXISTS user:42 email
HLEN key	Retourne le nombre de champs dans le hash.	HLEN user:42



Cas d'utilisation de HASH : un profil utilisateur

Le type Hash permet de stocker plusieurs champs (paires clé/valeur) sous une même clé Redis. Il est idéal pour représenter un objet avec plusieurs attributs (ex. profil utilisateur).

1 Le besoin

Notre application web doit stocker les informations d'un utilisateur :
nom, email, âge, rôle, date d'inscription, ...

Nous voulons :

- regrouper tous les attributs sous une seule clé Redis,
- pouvoir lire et modifier individuellement un champ,
- récupérer facilement l'ensemble du profil.



Alice
alice@example.com
25 ans
utilisateur

2 La structure

On utilise un HASH où :

- la clé est l'identifiant de l'utilisateur,
- chaque champ représente un attribut du profil avec sa valeur.

user:42	
Champ	Valeur
name	Alice
email	alice@example.com
age	25
role	utilisateur
created_at	2026-09-24
...	...

3 Exemple d'utilisation



Créer ou mettre à jour un profil utilisateur

```
HSET user:42 name "Alice" \  
email "alice@example.com" \  
age "25" \  
role "utilisateur" \  
created_at "2026-09-24"
```



Récupérer un champ spécifique

```
HGET user:42 email  
# → "alice@example.com"
```



Récupérer l'ensemble du profil

```
HGETALL user:42
```



Modifier un champ

```
HSET user:42 age "26"
```



Supprimer un champ

```
HDEL user:42 role
```

4 Résultat

```
127.0.0.1:6379> HGETALL user:42  
1) "name"  
2) "Alice"  
3) "email"  
4) "alice@example.com"  
5) "age"  
6) "25"  
7) "role"  
8) "utilisateur"  
9) "created_at"  
10) "2026-09-24"
```

Représentation du profil utilisateur

```
{  
  "name": "Alice",  
  "email": "alice@example.com",  
  "age": 25,  
  "role": "utilisateur",  
  "created_at": "2026-09-24"  
}
```



Pourquoi utiliser un HASH ?



Regroupe plusieurs attributs sous une même clé



Accès rapide à un champ spécifique



Mise à jour simple d'un ou plusieurs champs

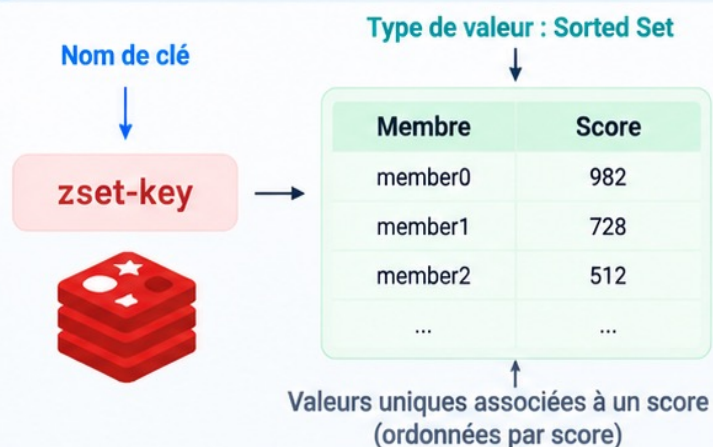


Adapté pour représenter des objets (utilisateur, produit, session, ...)

Redis – Structure de données : SORTED SET (ZSET)

La structure Sorted Set permet de stocker un ensemble de valeurs uniques associées à un score numérique. Les éléments sont automatiquement ordonnés selon leur score.

1 Concept



2 Ce qu'elle contient

Alice	1200
Bob	950
Charlie	870
David	400
...	...



- Chaque **membre** est unique.
- Chaque membre est associé à un **score** (nombre entier ou flottant).
- Les **éléments** sont automatiquement **ordonnés par score** (du plus petit au plus grand).

3 Capacités de lecture/écriture



Ajouter un membre avec un score
(si le membre existe déjà, son score est mis à jour)



Récupérer les membres dans l'ordre
(par position)



Récupérer les membres selon une plage de scores
(ex. : entre 100 et 1000)



Supprimer un membre
(s'il existe)



Obtenir le classement / le top N
(les meilleurs scores)

4 Commandes principales

Commande	Description	Exemple
<code>ZADD key score member [score member ...]</code>	Ajoute un membre avec un score (si le membre existe, son score est mis à jour).	<code>ZADD zset-key 728 member1</code>
<code>ZRANGE key start stop</code>	Récupère les membres dans l'ordre de leur score (du plus petit au plus grand).	<code>ZRANGE zset-key 0 2</code>
<code>ZRANGEBYSCORE key min max</code>	Récupère les membres dont le score est dans une plage donnée.	<code>ZRANGEBYSCORE zset-key 500 1000</code>
<code>ZREM key member [member ...]</code>	Supprime un ou plusieurs membres.	<code>ZREM zset-key member1</code>
<code>ZREVRANGE key start stop</code>	Récupère les membres dans l'ordre décroissant des scores (du plus grand au plus petit).	<code>ZREVRANGE zset-key 0 2</code>
<code>ZSCORE key member</code>	Récupère le score d'un membre.	<code>ZSCORE zset-key member1</code>



Cas d'utilisation de ZSET : un classement de joueurs

Le type Sorted Set (ZSET) permet de stocker un ensemble d'éléments uniques associés à un score. Il est idéal pour maintenir un classement, un top N ou des métriques ordonnées.

1 Le besoin

Notre application de jeu en ligne doit afficher le classement des joueurs selon leur score. Nous voulons :

- ajouter ou mettre à jour un score,
- récupérer le top 10,
- connaître la position d'un joueur.



2 La structure

On utilise un ZSET où :

- la clé (membre) est le nom du joueur,
- la valeur (score) est son score.

zset:leaderboard

Membre (joueur)	Score
Alice	1250
Charlie	1100
Bob	980
David	870
...	...

Classement (descendant)

3 Exemple d'utilisation

+ Ajouter ou mettre à jour un score

```
ZADD leaderboard 1250 "Alice"  
ZADD leaderboard 980 "Bob"  
ZADD leaderboard 1100 "Charlie"
```

🔍 Récupérer le top 3

```
ZREVRANGE leaderboard 0 2 WITHSCORES
```

👤 Obtenir la position d'un joueur

```
ZREVRANK leaderboard "Alice"
```

4 Résultat attendu

Top 3 (avec scores)

```
127.0.0.1:6379> ZREVRANGE leaderboard 0 2 WITHSCORES  
1) "Alice"  
2) "1250"  
3) "Charlie"  
4) "1100"  
5) "Bob"  
6) "980"
```

Position de « Bob »

```
127.0.0.1:6379> ZREVRANK leaderboard "Bob"  
(integer) 2
```

Mise à jour du score

```
127.0.0.1:6379> ZADD leaderboard 1500 "Bob"  
(integer) 1
```



Pourquoi utiliser un ZSET ?

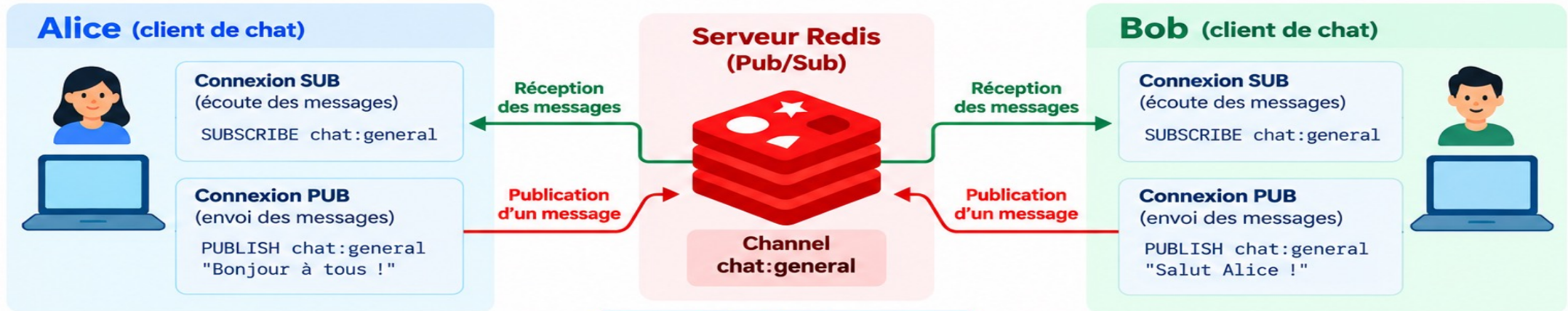
- ✓ **Maintien automatique de l'ordre**
(tri par score)
- ✓ **Accès rapide au top N**
(ex. : top 10, top 100)
- ✓ **Mise à jour simple des scores**
(pas besoin de recalculer le classement)
- ✓ **Élément unique**
(pas de doublons)



Le ZSET est la solution idéale pour tous les cas où vous avez besoin d'un ensemble ordonné par une valeur numérique.

Modèle Pub/Sub avec Redis – Exemple d'un chat

Les clients publient des messages dans un channel et tous les clients abonnés à ce channel les reçoivent en temps réel.



Exemple de flux de messages



Points clés

- SUBSCRIBE permet uniquement de **recevoir** les messages d'un ou plusieurs **channels**.
- PUBLISH permet de publier un message dans un **channel**.
- Publication et réception sont **indépendantes**.
- Un client peut publier sans être abonné et peut s'abonner sans jamais publier.



Côté client de chat

- Le client gère généralement deux connexions Redis (ou deux threads) :
 - une pour s'abonner et écouter les messages,
 - une autre pour publier les messages saisis par l'utilisateur.
- Lorsqu'un message est publié, tous les clients abonnés au channel le reçoivent, y compris l'émetteur.



À retenir

- Redis ne conserve pas les messages Pub/Sub : seuls les clients abonnés au moment de la publication reçoivent le message.
- Si un client est déconnecté, les messages publiés pendant son absence sont perdus.
- Pour conserver les messages et permettre un traitement ultérieur, on utilise Redis Streams.

TP 01 Redis

